

The Task Graph

Work is structure. A project without structure is a list—and lists lie. They hide the fact that you cannot deploy before you test, cannot test before you build, cannot build before you design. A list says “here are things to do.” A graph says “here is the order in which reality permits you to do them.”

Workgraph models work as a directed graph. Tasks are nodes. Dependencies are edges. The graph is the single source of truth for what exists, what depends on what, and what is available for execution right now. Everything else—the coordinator, the agency, the evolution system—reads from this graph and writes back to it. The graph is not a view of the project. It *is* the project.

Tasks as Nodes

A task is the atom of work. It has an identity, a lifecycle, and a body of metadata that guides both human and machine execution. Here is the anatomy:

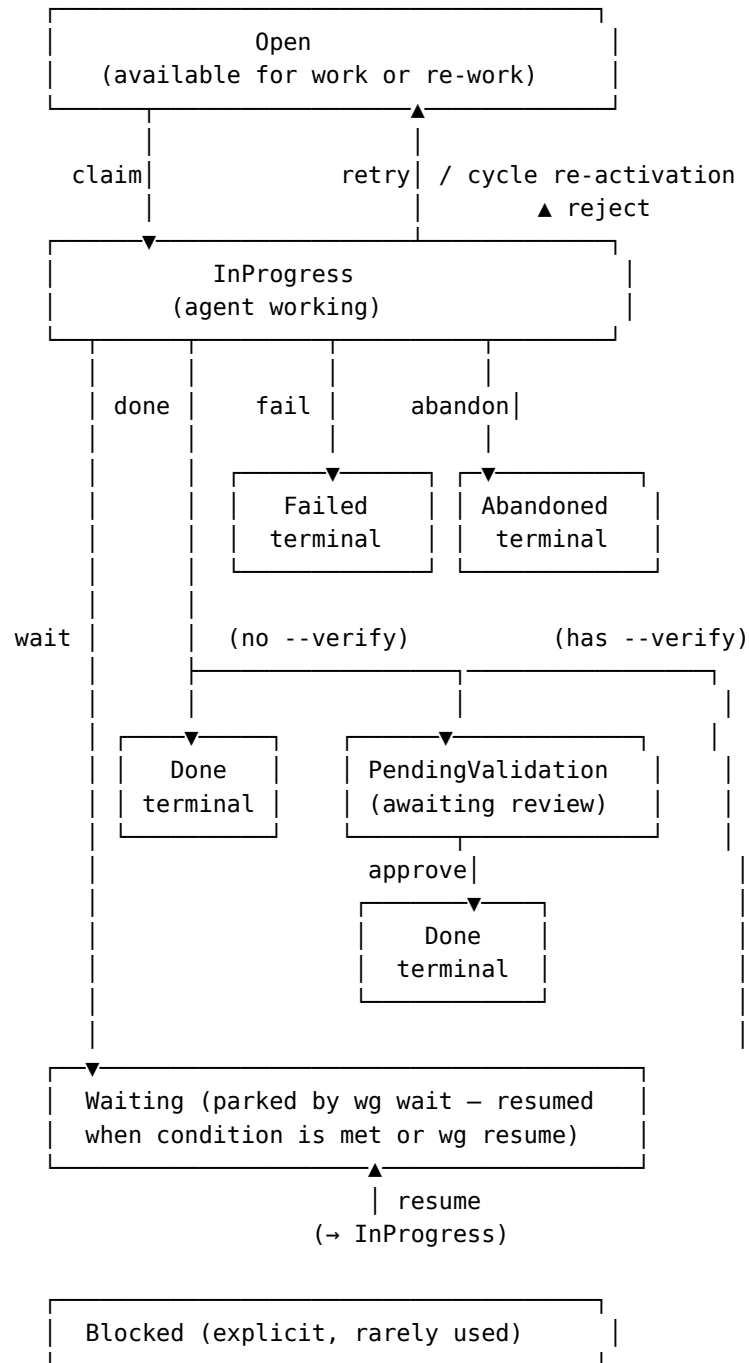
Field	Purpose
id	A slug derived from the title at creation time. The permanent key—used in every edge, every command, every reference. Once set, it never changes.
title	Human-readable name. Can be updated without breaking references.
description	The body: acceptance criteria, context, constraints. What an agent (human or AI) needs to understand the work.
status	Lifecycle state. One of eight values—see below.
estimate	Optional cost and hours. Used by budget fitting and forecasting.
tags	Flat labels for filtering and grouping.
skills	Required capabilities—matched against agent capabilities at dispatch time.
inputs	Paths or references the task needs to read.
deliverables	Expected outputs—what the task should produce.
artifacts	Actual outputs recorded after completion.
exec	A shell command for automated execution via the shell executor.
model	Preferred AI model (haiku, sonnet, opus). Overrides coordinator and agent defaults.
provider	LLM provider for this task (anthropic, openai, openrouter, local). Overrides coordinator and agent defaults.
exec_mode	Execution weight controlling the agent’s tool access. One of full (default—complete tool access), light (read-only tools), bare (only wg CLI), or shell (no LLM—runs the exec field directly).
verify	Verification criteria—if set, the task requires review before it can be marked done.
agent	Content-hash ID binding an agency agent identity to this task.
visibility	Controls what information crosses organizational boundaries during trace exports. One of internal (default—organization only), public (sanitized sharing without agent output or logs), or peer (richer detail for trusted peers, including evaluations and patterns).
context_scope	Controls how much context the agent receives in its prompt. One of clean (bare executor), task (default—workflow commands and graph patterns), graph (adds project description and 1-hop neighborhood), or full (adds complete graph summary and CLAUDE.md). Each tier is a strict superset of the one below. Overrides role and coordinator defaults when set.
delay	Duration to wait before the task becomes ready (e.g., 30s, 5m, 1h, 1d). Set via --delay on wg add/wg edit.
not_before	Absolute ISO 8601 timestamp before which the task will not be dispatched. Set via --not-before on wg add/wg edit.
log	Append-only progress entries with timestamps and optional actor attribution.

Table 1: Task fields. Every field except id, title, and status is optional.

Tasks are not just descriptions of work—they are self-contained dispatch packets. An agent spawned for a task receives the description, the inputs, the skills, the log history, and the artifacts of completed dependencies. Everything needed to begin work is encoded on the node itself or reachable through its edges.

Status and Lifecycle

A task moves through eight statuses. Most follow the happy path; some take detours.



Listing 1: Task state machine. The eight statuses include three terminals (Done, Failed, Abandoned) that unblock dependents, plus PendingValidation and Waiting as non-terminal intermediate states.

Open is the starting state. A task is open when it has been created and is potentially available for work—though it may not yet be *ready* (a distinction explored below).

InProgress means an agent has claimed the task and is working on it. The coordinator sets this atomically before spawning the agent process.

Done, **Failed**, and **Abandoned** are the three *terminal* statuses. A terminal task will not progress further without explicit intervention—retry, manual re-open, or cycle re-activation. The crucial

design choice: all three terminal statuses unblock dependents. A failed upstream does not freeze the graph. The downstream task gets dispatched and can decide for itself what to do about a failed dependency—inspect the failure reason, skip the work, or adapt.

PendingValidation is entered when an agent calls `wg done` on a task that has `verify` criteria. The task is not yet terminal—it awaits external review. `wg approve` transitions it to Done; `wg reject` transitions it back to Open for re-dispatch (with the assignment cleared). After `max_rejections` (default: 3), rejection transitions the task to Failed instead.

Waiting means an agent has voluntarily parked the task via `wg wait`. The task is not terminal and will not be re-dispatched. It resumes (returning to InProgress) when its wait condition is met—a timer elapses, a dependent task reaches a target status, a message arrives, or a file changes—or when a human runs `wg resume`.

Blocked exists as an explicit status but is rarely used. In practice, a task is *blocked* when its `after` list contains non-terminal entries—this is a derived condition, not a declared status. The explicit Blocked status is a manual override for cases where a human wants to freeze a task for reasons outside the graph.

Terminal Statuses Unblock: A Design Choice

This merits emphasis. In many task systems, a failed dependency blocks everything downstream until a human intervenes. Workgraph takes the opposite stance: failure is information, not obstruction.

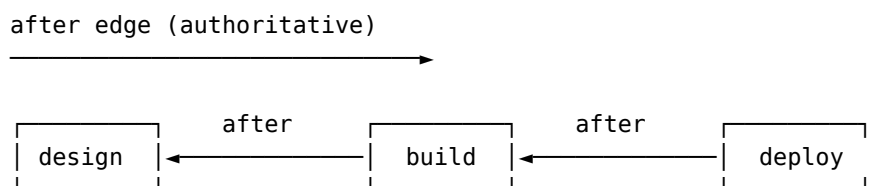
When task A fails and task B depends on A, B becomes ready. B’s agent receives context from A—the failure reason, the log entries, the artifacts (if any). The agent can then decide: retry the work itself, produce a partial result, or fail explicitly with its own reason. The graph keeps moving.

This works because terminal means “this task has reached an endpoint for this iteration.” Done is a successful endpoint. Failed is an unsuccessful one. Abandoned is a deliberate withdrawal. In all three cases, the task is no longer going to change, so dependents can proceed with whatever information is available.

The alternative—frozen pipelines waiting for human intervention—violates the principle that the graph should be self-advancing. If you need a hard stop on failure, model it explicitly: add a guard condition or a verification step. Don’t rely on the scheduler to enforce business logic through status propagation.

Dependencies: after and before

Dependencies are directed edges expressing temporal ordering. Task B depends on task A means: B cannot be ready until A reaches a terminal status. This is expressed by placing A’s ID in B’s `after` list—B comes *after* A.



Read as: build is after design. deploy is after build.
Equivalently: design is before build. build is before deploy.

Listing 2: Dependency edges. `after` is authoritative; `before` is its computed inverse.

The after list is the source of truth. The before list is its inverse, maintained for bidirectional traversal—if B is after A, then A's before list includes B. The scheduler never reads before; it only checks after. The inverse is a convenience index for commands like `wg impact` and `wg bottlenecks` that need to traverse the graph forward from a task to its dependents.

Transitivity works naturally. If C is after B and B is after A, then C cannot be ready while A is non-terminal, because B cannot be ready (and thus cannot become terminal) while A is non-terminal. No transitive closure computation is needed—the scheduler checks each task's immediate predecessors, and the chain resolves itself one link at a time.

A subtlety: if a task references a predecessor that does not exist in the graph, the missing reference is treated as resolved. This is a fail-open design—a dangling reference does not freeze the graph. The `wg check` command flags these as warnings, but the scheduler proceeds.

Readiness

A task is *ready* when four conditions hold simultaneously:

1. **Open status.** The task must be in the Open state. Tasks that are in-progress, done, failed, abandoned, or explicitly blocked are never ready.
2. **Not paused.** The task's paused flag must be false. Pausing is an explicit hold—the task retains its status and all other state, but the coordinator will not dispatch it.
3. **Past time constraints.** If the task has a `not_before` timestamp, the current time must be past it. If the task has a `ready_after` timestamp (set by cycle delays), the current time must be past that too. Invalid or missing timestamps are treated as satisfied—they do not prevent readiness.
4. **All predecessors terminal.** Every task ID in the after list must correspond to a task in a terminal status (done, failed, or abandoned). Non-existent predecessors are treated as resolved.

These four conditions are evaluated by `ready_tasks()`, the function that the coordinator calls every tick to find work to dispatch. Ready is a precise, computed property—not a flag someone sets. You cannot manually mark a task as ready; you can only create the conditions under which the scheduler derives it.

The `not_before` field enables future scheduling: “do not start this task before next Monday.” The `ready_after` field serves a different purpose—it is set automatically by cycle delays, creating pacing between cycle iterations. Both are checked against the current wall-clock time.

Structural Cycles: Intentional Iteration

Workgraph is a directed graph, not a DAG. This is a deliberate design choice.

Most task systems are acyclic by construction—dependencies flow in one direction, and cycles are errors. This works for projects that execute once: design, build, test, deploy, done. But real work is often iterative. You write a draft, a reviewer reads it, you revise based on feedback, the reviewer reads again. A CI pipeline builds, tests, and if tests fail, loops back to build with fixes. A monitoring system checks, investigates, fixes, verifies, and then checks again.

These patterns are cycles, and they are not bugs. They are the structure of iterative work.

Workgraph makes them first-class through *structural cycles*—cycles that emerge naturally from after edges in the task graph, detected automatically by the system.

How Structural Cycles Work

A structural cycle is a set of tasks whose after edges form a cycle. If task A is after task C, task C is after task B, and task B is after task A, the system detects this cycle automatically using Tarjan's SCC (strongly connected component) algorithm. No special edge type is needed—the cycle is a structural property of the graph.

Each cycle has a *header*: the entry point, identified as the task with predecessors outside the cycle. The header carries a `CycleConfig` that controls iteration:

Field	Purpose
<code>max_iterations</code>	Hard cap on how many times the cycle can iterate. Mandatory—no unbounded cycles.
<code>guard</code>	A condition that must be true for the cycle to iterate. Optional—if absent, the cycle iterates unconditionally (up to <code>max_iterations</code>).
<code>delay</code>	Optional duration (e.g., "30s", "5m", "1h") to wait before the next iteration. Sets the header's <code>ready_after</code> timestamp.
<code>no_converge</code>	When set, agents cannot signal early convergence via <code>--converged</code> . All iterations (up to <code>max_iterations</code>) are forced to run. Set via <code>--no-converge</code> on <code>wg add/wg edit</code> .
<code>restart_on_failure</code>	Whether to automatically restart the cycle when a member fails (default: true). Disabled via <code>--no-restart-on-failure</code> . The <code>max_failure_restarts</code> field caps the number of failure-triggered restarts (default: 3).

Table 2: `CycleConfig` fields on the cycle header task. Every configured cycle requires a `max_iterations` cap.

The critical insight: the cycle header receives a *back-edge exemption* in the readiness check. Normally, a task is waiting when any of its after predecessors is non-terminal. But the header's predecessors within the cycle (the back-edges) are exempt—this allows the header to become ready on the first iteration even though its cycle predecessors have not yet completed. Non-header tasks in the cycle still wait for their predecessors normally, so the cycle executes in order from the header through the body.

A cycle without a `CycleConfig` on any member is flagged by `wg check` as an unconfigured deadlock—it will not iterate and the header will not receive back-edge exemption.

Guards

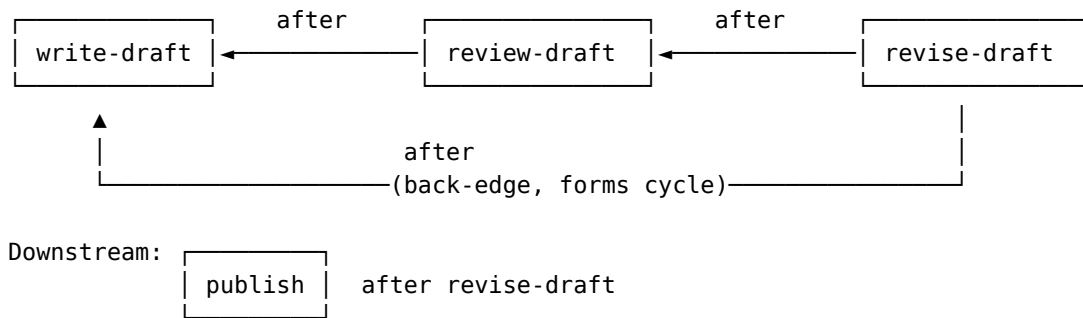
A guard is a condition on a cycle's `CycleConfig` that controls whether the cycle iterates. Three kinds:

- **Always.** The cycle iterates unconditionally on every completion, up to `max_iterations`. Used for monitoring loops and fixed-iteration patterns.
- **TaskStatus.** The cycle iterates only if a named task has a specific status. The classic use: "iterate back to writing if the review task failed." This is the mechanism for conditional retry.
- **IterationLessThan.** The cycle iterates only if the header's iteration count is below a threshold. Redundant with `max_iterations` in simple cases, but explicit when you want the guard condition visible in the graph data.

If no guard is specified, the cycle behaves as *Always*—it iterates on every completion up to the iteration cap.

A Review Cycle, Step by Step

Consider a three-task review cycle:



write-draft has CycleConfig: max_iterations=5,
guard=task:review-draft=failed

Listing 3: A structural cycle. All edges are after edges. The back-edge from write-draft to revise-draft creates the cycle.

The cycle is detected automatically: write-draft → review-draft → revise-draft → write-draft. The header is write-draft (it has external predecessors or is the entry point). Its CycleConfig sets max_iterations: 5 and a guard condition.

Created with:

```

wg add "write-draft" --max-iterations 5 --cycle-guard "task:review-draft=failed"
wg add "review-draft" --after write-draft
wg add "revise-draft" --after review-draft
wg add "publish" --after revise-draft

```

Then create the back-edge that forms the cycle:

```

wg edit write-draft --add-after revise-draft

```

Here is the execution:

1. write-draft is the cycle header. Its back-edge predecessor (revise-draft) is exempt from the readiness check. It is ready. The coordinator dispatches an agent.
2. The agent completes the draft and calls `wg done write-draft`. The task becomes terminal.
3. review-draft has all predecessors terminal (just write-draft). It becomes ready. The coordinator dispatches a reviewer agent.
4. The reviewer finds problems and calls `wg fail review-draft --reason "Missing section 3"`. The task is now terminal (failed).
5. revise-draft has all predecessors terminal (review-draft is failed—and failed is terminal). It becomes ready. The coordinator dispatches an agent.
6. The agent reads the failure reason from review-draft, revises accordingly, and calls `wg done revise-draft`.
7. All cycle members are now terminal. The system evaluates cycle iteration: the guard checks review-draft's status—it is Failed. The header's loop_iteration is 0, below max_iterations (5). The cycle iterates.
8. All cycle members are re-opened: status set to Open, assignments and timestamps cleared, loop_iteration incremented to 1. A log entry records: "Re-activated by cycle iteration (iteration 1/5)."
9. write-draft is again ready (back-edge exemption). The cycle begins again.

If the reviewer eventually approves (calls `wg done review-draft` instead of `wg fail`), then when all members complete, the guard checks review-draft's status—it is Done, not Failed. The guard

condition is not met. The cycle does not iterate. All members stay done. `publish` has all predecessors terminal. The graph proceeds.

Cycle Re-Opening

When a cycle iterates, *all* cycle members are re-opened simultaneously. The system knows exactly which tasks belong to the cycle through SCC analysis—no BFS traversal needed. Every member's status is set to `Open`, its assignment and timestamps are cleared, and its `loop_iteration` is incremented to match the new iteration count.

This ensures the entire cycle is available for re-execution, and every member's status accurately reflects the cycle state.

Bounded Iteration

Every cycle header must specify `max_iterations` in its `CycleConfig`. There are no unbounded cycles. When the header's `loop_iteration` reaches the cap, the cycle stops iterating, regardless of guard conditions. All members stay done. Downstream work proceeds.

This is a safety property. A guard condition with a logic error could iterate indefinitely; `max_iterations` guarantees that every cycle terminates.

Early Convergence

The iteration cap is a ceiling, not a target. In practice, iterative work often converges before the maximum is reached—a refine agent determines the output is stable, a review cycle approves on the third pass instead of the fifth, a monitoring check finds the system healthy. Running all remaining iterations after convergence wastes compute and delays downstream work.

Any agent working on a cycle member can signal convergence by running `wg done <task-id> --converged`. This marks the task as done and adds a "converged" tag to the *cycle header* (regardless of which member the agent completes). When the cycle evaluator checks whether to iterate, it sees the tag on the header and stops—the cycle does not iterate, regardless of guard conditions or remaining iterations. Downstream tasks proceed immediately.

The convergence tag is durable but not permanent. Running `wg retry` on a converged task clears the tag along with resetting the task to open, so the cycle can iterate again if needed. This means convergence is an agent's assertion about *this* iteration's outcome, not a permanent lock on the cycle structure.

The coordinator supports this mechanism in the dispatch cycle: when rendering a prompt for a task that is part of a structural cycle, it includes a note about the `--converged` flag, informing the agent that early termination is available. The agent decides—the system does not guess.

Cycle Delays

A cycle's `CycleConfig` can specify a delay: a human-readable duration like "30s", "5m", "1h", or "1d". When a delayed cycle iterates, instead of making the header immediately ready, it sets the header's `ready_after` timestamp to `now + delay`. The scheduler will not dispatch the header until the delay has elapsed.

This creates pacing between iterations. A monitoring cycle that checks system health every five minutes uses a delay of "5m". A review cycle that gives the author time to revise before the next review might use "1h".

Pause and Resume

Sometimes you need to stop a cycle—or any task—without destroying its state. The paused flag provides this control.

`wg pause <task>` sets the flag. The task retains its status, its cycle iteration count, its log entries—everything. But the scheduler will not dispatch it. It is invisible to `ready_tasks()`.

`wg resume <task>` clears the flag. The task re-enters the readiness calculation. If it meets all four readiness conditions, it becomes available for dispatch on the next coordinator tick.

Pausing is orthogonal to status. You can pause an open task to hold it. You can pause a task mid-cycle to halt iteration without losing state. When you resume, the cycle picks up where it left off.

Placement Hints

When a new task is added, the coordinator can automatically position it in the dependency graph through *placement*—an optional feature controlled by `wg config --auto-place`. Placement hints on `wg add` guide this positioning:

- `--no-place` skips automatic placement entirely, leaving the task with only the dependencies explicitly specified via `--after`.
- `--place-near <IDS>` suggests placing the task near the specified tasks in the graph—useful for grouping related work.
- `--place-before <IDS>` suggests inserting the task before the specified tasks, adding dependency edges so those tasks come after the new one.

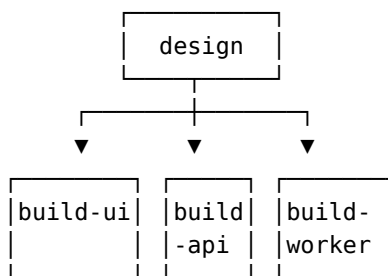
Placement is a convenience, not a constraint. All dependency edges it creates are ordinary `after` edges, visible in the graph and editable with `wg edit`. If placement produces an undesirable result, adjust the edges manually.

Emergent Patterns

The dependency edges (`after`/`before`) and structural cycles are the only primitives. But from these mechanisms, several structural patterns emerge naturally.

Fan-Out (Map)

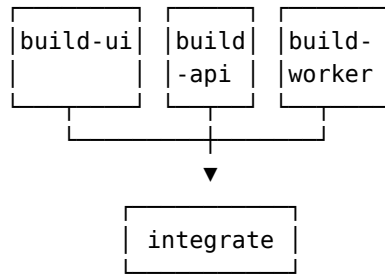
One task is before several children. When the parent completes, all children become ready simultaneously and can execute in parallel.



Listing 4: Fan-out: one parent completes, enabling parallel children.

Fan-In (Reduce)

Several tasks are before a single aggregator. The aggregator becomes ready only when all of its predecessors are terminal.



Listing 5: Fan-in: multiple parents must all complete before the child is ready.

Combined, fan-out and fan-in produce the *map/reduce pattern*: a coordinator task fans out parallel work, then an aggregator task fans in the results. This is not a built-in primitive. It arises naturally from the shape of the dependency edges.

Pipelines

A linear chain: B is after A, C is after B, D is after C. Each task becomes ready only when its single predecessor completes. Pipelines are the simplest dependency structure—a sequence.

Review Cycles

A dependency chain with a back-edge creating a structural cycle, as described above. The cycle executes repeatedly until a guard condition breaks it, convergence is signaled, or the iteration cap is reached. Review cycles are the canonical example of intentional iteration.

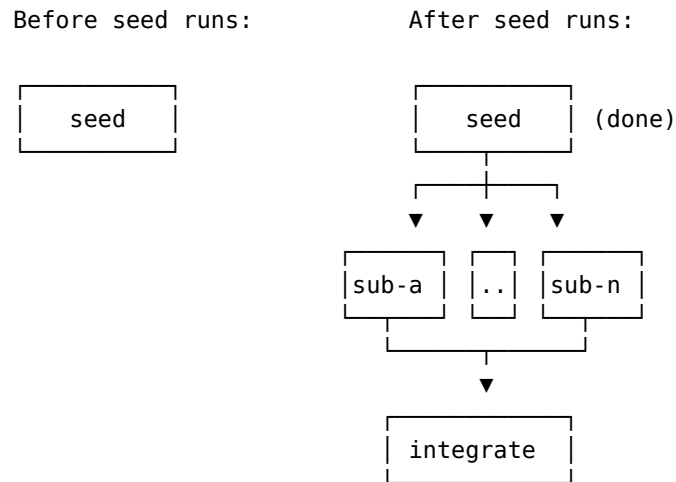
Functions: Reusable Patterns

When a workflow pattern proves useful—a review cycle that consistently produces good results, a map/reduce pipeline tuned for a particular domain—it can be extracted from a completed trace into a reusable template called a *function*. The `wg func extract` command takes a completed task and its subgraph, captures the task structure, dependencies, structural cycles, and guards, and parameterizes the variable parts: feature names, file paths, descriptions, and thresholds become named input variables. The result is stored as YAML in `.workgraph/functions/`.

Applying a function with `wg func apply` reverses the process. It takes a function name and a set of input values, substitutes them into the template, and creates concrete tasks in the graph with proper dependency wiring. The original pattern’s structure is preserved—its fan-out topology, its cycle bounds, its guard conditions—but applied to new work. Functions can also be shared across projects: the `--from` flag accepts a peer name or file path, enabling teams to import proven workflows from one another.

Seed Tasks (Generative Tasks)

A *seed task* is a task whose primary purpose is to bootstrap a subgraph—it fans out into subtasks that did not exist before it ran. The seed does not do the “real” work itself; it analyzes a problem, decomposes it into concrete steps, and creates the tasks that perform those steps. Once the seed completes, the graph has new structure that the coordinator dispatches.



Listing 6: A seed task creates structure. The graph before execution has one node; the graph after has many.

The seed pattern is common in practice:

- A planning task that reads a spec, identifies components, and creates one implementation task per component.
- A research task that surveys a topic, identifies sub-questions, and creates investigation tasks for each.
- A triage task that reads an incoming report and creates the appropriate response tasks.

Seed tasks are often the root of a diamond (fan-out then fan-in) or scatter-gather topology. What distinguishes a seed from an ordinary fan-out parent is that the children *do not exist in the graph until the seed runs*. The graph is not just executed—it is grown.

In theoretical terms, a seed task is a *generative task*: it produces the network components that constitute the next phase of work. This connects to Maturana and Varela’s concept of autopoiesis—a production relation where the system produces its own components. The seed is the autopoietic act at the task level: a node that produces nodes.

Casually, seed tasks are sometimes called *spark tasks*—the spark that ignites a subgraph into existence.

Graph Analysis

Workgraph provides several analysis tools that read the graph structure and compute derived properties. These are instruments, not concepts—they report on the graph rather than define it.

Critical path. The longest dependency chain among active (non-terminal) tasks, measured in estimated hours. The critical path determines the minimum time to completion—no amount of parallelism can shorten it. Tasks on the critical path have zero slack; delays to any of them delay the entire project. `wg critical-path` computes this, skipping cycles to avoid infinite traversals.

Bottlenecks. Tasks that transitively block the most downstream work. A bottleneck is not necessarily on the critical path—it might block many short chains rather than one long one. `wg bottlenecks` ranks tasks by the count of transitive dependents, providing recommendations for tasks that should be prioritized.

Impact. Given a specific task, what depends on it? `wg impact <task>` traces both direct and transitive dependents, computing the total hours at risk if the task is delayed or fails.

Cost. The total estimated cost of a task including all its transitive dependencies, computed with cycle detection to avoid double-counting shared ancestors in diamond patterns.

Forecast. Projected completion date based on remaining work, estimated velocity, and dependency structure.

Visualization. `wg viz` renders the graph as text. The `--graph` format produces a 2D spatial layout using Unicode box-drawing characters, positioning tasks by their dependency depth—roots at the top, leaf tasks at the bottom. Nodes are color-coded by status and connected by vertical lines that split at fan-out points and merge at fan-in points. The layout algorithm assigns layers via topological sort, then orders nodes within each layer to minimize edge crossings.

These tools share a common pattern: they traverse the graph using *after* edges (and their *before* inverse), respect the visited-set pattern to handle cycles safely, and report on the structure without modifying it.

Storage

The graph is stored as JSONL—one JSON object per line, one node per object. A graph file might look like this:

```
{"kind":"task","id":"write-draft","title":"Write draft","status":"open","after":
["revise-draft"],"cycle_config":{"max_iterations":5,"guard":{"TaskStatus":
{"task":"review-draft","status":"failed"}}}}
{"kind":"task","id":"review-draft","title":"Review draft","status":"open","after":
["write-draft"]}
{"kind":"task","id":"revise-draft","title":"Revise","status":"open","after":["review-
draft"]}
{"kind":"task","id":"publish","title":"Publish","status":"open","after":["revise-
draft"]}
```

Listing 7: A graph file in JSONL format. Each line is a self-contained node.

JSONL has three virtues for this purpose. It is human-readable—you can inspect and edit it with any text editor. It is version-control-friendly—adding or modifying a task changes one line, producing clean diffs. And it supports atomic writes with file locking—concurrent processes cannot corrupt the graph because every write acquires an exclusive lock, rewrites the file, and releases.

The graph file lives at `.workgraph/graph.jsonl` and is the canonical state of the project. There is no database, no server dependency. Everything reads from and writes to this file. The service daemon, when running, holds no state beyond what the file contains—it can be killed and restarted without loss.

Alongside the graph file, the operations log (`operations.jsonl`) records every mutation: task creation, status changes, dependency additions, cycle iterations, evaluations. This log is the project’s trace—its organizational memory. The `wg trace` command queries it. `wg trace export` produces a filtered, shareable snapshot with visibility controls: an *internal* export includes everything, a *public* export sanitizes (omitting agent output and logs), and a *peer* export provides richer detail for trusted collaborators. `wg trace import` ingests a peer’s export, enabling cross-boundary knowledge transfer. The graph file tells you where the project *is*. The operations log tells you how it got there.

—

The task graph is the foundation. Dependencies (via *after* edges) encode the ordering constraints of reality. Structural cycles encode the iterative patterns of practice. Readiness is a derived property—the scheduler’s answer to “what can happen next?” The coordinator uses this answer to dispatch

work, as described in the section on coordination and execution. The agency system uses the graph to record evaluations at each task boundary, as described in the section on evolution.

A well-designed task graph does not just organize work. It makes the structure of the project legible—to humans reviewing progress, to agents receiving dispatch, and to the system itself as it learns from its own history.